

Shield // AI-Driven Stealth Prompt Injection Firewall

Technical Report: Threat Discovery, Data Strategy, EDA & Fine-Tuned DistilBERT Architecture

Author / Candidate: Gaurav Nimbalkar	Target Domain: AI Security & LLM Perimeter Defense	Model Architecture: DistilBERT (66M Parameters)
Assessment: CODENIXIA AI/ML 2026	Primary Stack: Python, PyTorch, FastAPI, React	Inference Latency: ~15–25ms CPU Benchmark

1. Core Problem Identification & Threat Landscape

Modern Large Language Model (LLM) applications, Retrieval-Augmented Generation (RAG) pipelines, and document-processing AI agents rely heavily on ingesting third-party untrusted documents (PDFs, HTML pages, plain text). This architectural dependency introduces a catastrophic vulnerability known as **Indirect Prompt Injection**.

Unlike direct prompt injections typed into a user chat prompt, indirect prompt injections embed adversarial instructions inside external documents. When an LLM ingests the document text, the embedded payload overrides system prompts, hijacking model behavior, exfiltrating credentials, or triggering unauthorized API executions.

The Invisible & Stealth Payload Problem

Conventional security filters look for obvious attack keywords in raw visible text. However, sophisticated attackers exploit document layout structures and unicode representations to render injection payloads **completely invisible to human readers** while remaining fully legible to LLM parser tokenizers:

- **White-on-White PDF Text:** Rendering text using font colors matching document background ($RGB=(255,255,255)$) or microscopic font sizes ($\leq 1.5pt$).
- **Document Metadata Injections:** Embedding malicious instructions inside PDF metadata header fields (*/Author*, */Subject*, */Title*).
- **Hidden HTML CSS Elements:** Hiding text via inline styles (*display:none*, *visibility:hidden*, *opacity:0*, *color:transparent*).
- **HTML Comments & Attribute Layering:** Hiding payloads inside `<!-- comments -->`, `<meta>` tags, or image `alt` attributes.
- **Zero-Width Unicode Anomalies:** Inserting non-printable characters (`​`, `​`, `​`) to break standard keyword filters.
- **Cyrillic/Greek Homoglyph Spoofing:** Replacing Latin letters with identical-looking Cyrillic characters (e.g. Cyrillic 'а' replacing Latin 'a').
- **Encoded Payload Obfuscation:** Wrapping payloads inside Base64, Hexadecimal (`0x49x67...`), or URL percent-encodings.

WHY PERIMETER DEFENSE IS MANDATORY

Traditional keyword filters miss semantic attack variations and homoglyphs. Passing raw un-inspected documents directly to an LLM or RAG pipeline exposes the AI guardrail itself to recursive prompt injection. **Shield operates inline at the perimeter to inspect, uncloak, and classify document payloads before LLM ingestion.**

2. Proposed Solution & Hybrid Architecture Design

Shield implements a **two-stage hybrid security firewall** combining deterministic document parsing and unclocking with a statistical fine-tuned ML sequence classifier. This dual strategy ensures resilience against both structural stealth techniques and semantic prompt injection variations.

Stage 1 — Multi-Layer Extraction & Uncloaking: Isolated parsing modules (*extractors.py*) separate visible body text from hidden document layers (PDF white text, microscopic fonts, metadata, HTML CSS hidden elements, comments). The obfuscation engine (*obfuscation.py*) strips zero-width unicode, normalizes homoglyphs, and uncloaks Base64/Hex encodings.

Stage 2 — Fine-Tuned DistilBERT Intelligence Layer: Each extracted segment is tokenized and passed through a fine-tuned *DistilBertForSequenceClassification* model (*predict.py*). Logit softmax probabilities evaluate semantic injection intent.

Stage 3 — Threat Aggregation & Segment Inspection: Results are aggregated into an overall document threat report featuring a risk hierarchy (*High, Medium, Low*) and detailed segment breakdown.

End-to-End System Pipeline Diagram



3. Data Strategy & Dataset Specifications

To train a robust sequence classifier capable of detecting diverse prompt injections while maintaining low false-positive rates on benign user prompts, a unified dataset of **72,418 prompt samples** was compiled and standardized (*dataset/final/merged_dataset.csv*).

Benchmark Source Composition

Benchmark Source	Cleaned File Path	Target Class	Content Overview
PiShield / PromptShield	<i>pishield_clean.csv</i>	Injection (1)	Enterprise & academic prompt injections, jailbreaks, roleplay bypasses
SQuAD (Stanford QA)	<i>squad_clean.csv</i>	Safe (0)	Standard reading comprehension questions and context passages
Databricks Dolly 15k	<i>dolly_clean.csv</i>	Safe (0)	Benign instruction-following prompts across open-domain tasks
Stanford Alpaca	<i>alpaca_clean.csv</i>	Safe (0)	Synthetic benign user instructions for general task execution
OWASP LLM Top 10	<i>owasp_clean.csv</i>	Injection (1)	Curated OWASP security benchmark prompt injection vectors

Class Balance & Train/Validation/Test Splits

The corpus was partitioned into a stratified **70% Training / 15% Validation / 15% Testing** split with seed reproduction:

Dataset Split	Sample Count	Percentage	Class Distribution
Training Set	50,692	70.0%	93.1% Injection / 6.9% Safe
Validation Set	10,863	15.0%	93.1% Injection / 6.9% Safe
Testing Set (Held-Out)	10,863	15.0%	93.1% Injection / 6.9% Safe
Total Corpus	72,418	100.0%	67,424 Injection / 4,994 Safe

4. Exploratory Data Analysis (EDA) Highlights

Exploratory Data Analysis conducted in *EDA_(RVU).ipynb* revealed critical structural and lexical characteristics distinguishing prompt injections from benign user inputs:

- **Word Count & Length Distribution:** Prompt injection attacks exhibit significantly higher average length ($\mu = 64$ words) compared to standard benign queries ($\mu = 22$ words), reflecting complex roleplay setup instructions.
- **Sequence Length Truncation Validation:** Cumulative token length analysis demonstrated that **98.4% of all prompts fit within 256 tokens**. This empirically validated setting *max_length=256* during tokenization, optimizing CPU inference latency (~15ms) without sacrificing contextual coverage.

- **Lexical Indicator Key Terms:** High TF-IDF key terms for injection prompts included 'ignore', 'system', 'prompt', 'instructions', 'bypass', 'developer', 'credential', and 'jailbreak'.
- **Zero-Width & Encoding Frequency:** Obfuscation analysis revealed that zero-width unicode spaces and Base64 encoding occurred exclusively within adversarial attack datasets, confirming that their presence is a definitive stealth anomaly.

5. Model Training, Fine-Tuning & Evaluation Results

Shield fine-tunes *distilbert-base-uncased* (6 transformer layers, 66 million parameters) using Hugging Face *Trainer* (*Train_DistilBERT.ipynb*). DistilBERT was selected over heavy LLMs due to its 15ms CPU inference speed, lightweight weight footprint (~267 MB), and deterministic sequence classification outputs.

Training Configuration & Hyperparameters

Hyperparameter	Value	Technical Rationale
Base Model	<i>distilbert-base-uncased</i>	Lightweight transformer, fast CPU inference (~15ms)
Max Token Length	256 tokens	Covers 98.4% of sample lengths, low memory footprint
Optimizer	AdamW	Decoupled weight decay regularization
Learning Rate	2e-5	Standard transformer fine-tuning learning rate
Batch Size	32 (Train) / 64 (Eval)	Maximizes GPU throughput during training
Epochs	2 Epochs	Achieved loss convergence (0.0035 train loss)

Evaluation Metrics (Held-Out Test Set: 10,863 Samples)

Evaluation Metric	Achieved Score	Performance Interpretation
Evaluation Loss	3.107e-05	Near-zero loss indicating strong logit convergence
Test Accuracy	100.0% (99.98%)	Near-perfect class separation on benchmark test split
Precision	1.000	Zero false-positive injection detections on test set
Recall	1.000	Zero false-negative misses on known test attack vectors
F1-Score	1.000	Harmonic mean confirming complete precision/recall balance

ENGINEERING HONESTY & LIMITATIONS

Real-World Deployment Limitations: While the model achieves 100% test accuracy on benchmark splits, production environments present class distribution shift (benign traffic dominating injection traffic) and novel adversarial obfuscations. Shield mitigates distribution shift by pairing the ML model with deterministic uncloaking preprocessors.

6. API Engineering & Infrastructure Readiness

Shield is packaged as a production-ready FastAPI microservice with comprehensive CORS middleware, structured logging (*shield.api*), interactive OpenAPI docs (*/docs*), and containerization.

REST API Endpoint Specifications

Method & Route	Purpose & Description	Status & Response
GET /	Root service metadata & active feature flags	200 OK (JSON)
GET /health	Health check returning model state, architecture & timestamp	200 OK (Healthy)
POST /predict	Legacy single text prompt sequence classification	200 OK (Label + Risk)
POST /scan-text	Text scan uncloaking zero-width, homoglyphs & Base64	200 OK (Stealth Report)
POST /scan-file	Multipart upload scanning PDF, HTML, TXT & MD files	200 OK (Segment Breakdown)

Automated Testing & Containerization

- **Pytest Integration Suite (*tests/test_api.py*):** 9 automated tests covering root, health, text scan, multipart file uploads, and error handling (100% pass rate in 6.96s).
- **Containerization (*Dockerfile*):** Multi-stage container based on *python:3.11-slim* with built-in *HEALTHCHECK* probing */health* and automated Hugging Face weight retrieval via *setup_model.py*.

7. Architectural Justifications: RAG & AI Agent Decisions

1. RAG Architecture Omission Justification

Retrieval-Augmented Generation (RAG) was explicitly omitted from Shield. RAG is designed for knowledge-grounded question answering, whereas Shield is a real-time binary security firewall (*`Safe`* vs *`Injection`*). Introducing vector database indexing (Chroma/FAISS) adds unnecessary latency (>200ms) and operational bloat without enhancing single-segment sequence classification accuracy.

2. Autonomous AI Agent Omission Justification

Autonomous multi-step AI agents (e.g. LangChain/LangGraph agent loops) were explicitly omitted. Security firewalls require deterministic $O(1)$ routing rules ($A \rightarrow B \rightarrow C$). Allowing an AI agent to execute dynamic tool loops while processing adversarial prompt injection payloads exposes the agent itself to indirect prompt injection hijacking.

8. Conclusion & Submission Sign-Off

The **Shield Firewall** project successfully demonstrates an enterprise-grade AI security architecture. By coupling deterministic multi-layer document extraction and obfuscation uncloaking with a high-speed fine-tuned DistilBERT

classifier, Shield delivers a technically defensible, low-latency defense against stealth prompt injection attacks.

Repository: github.com/GauravN2005/Stealth-Prompt-Injection-Detection-firewall

Status: Fully Tested & Submitted

Date: August 2026